

Ingress & Traffic Management

Traefik, TLS & certificates, Gateway API, load balancing, reverse proxy, egress, and best practices — on one k3s node.

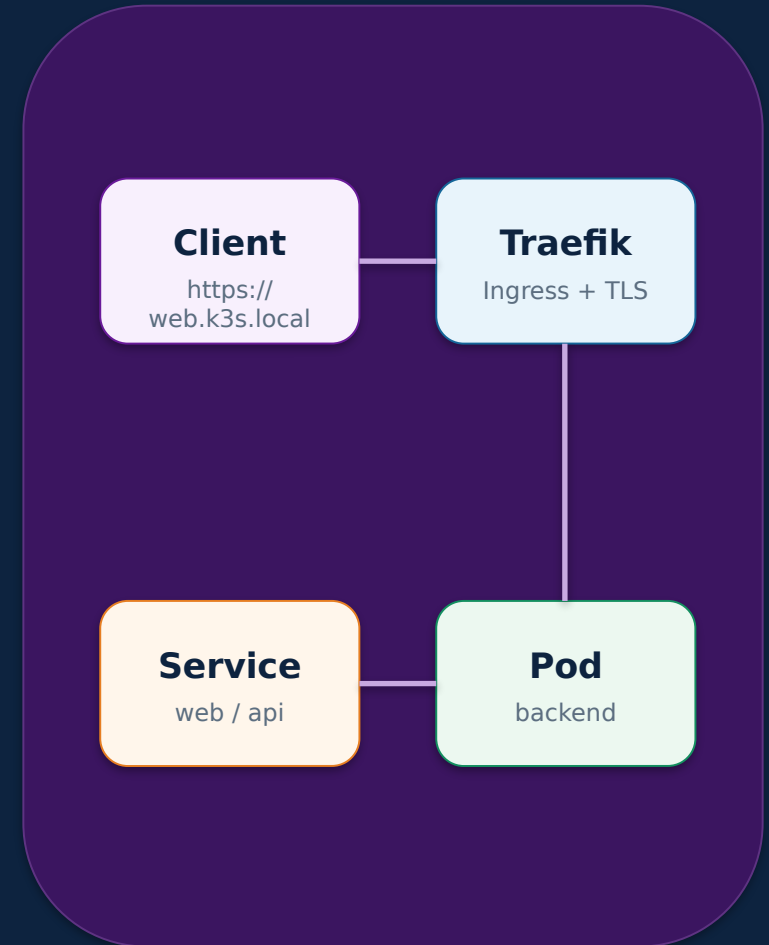
TRAEFIK

TLS

GATEWAY API

MIDDLEWARE

EGRESS



Where this sits in the course

Day 01

Workloads, config,
Secrets, RBAC.

Day 02

ClusterIP, NodePort,
ServiceLB,
ExternalName, headless,
NetworkPolicy.

Day 03

iptables, Flannel,
CoreDNS —
what's underneath Day
02.

Today

One entry point, many
hosts/paths,
TLS, and traffic-shaping
rules.

The question today answers

NodePort/LoadBalancer get a request to ONE Service. How do ten web apps share one IP and one set of ports 80/443, with TLS, host by host?

```
kubectl get ingressclass
```

```
kubectl get svc traefik -n kube-system
```

Single-node reality check

Everything here runs on ONE node. Two topics genuinely cannot be shown live — called out, not hidden.

Public ACME / Let's Encrypt certs

Needs a public DNS name + inbound 80/443 from the internet. This lab has neither.
Hands-on uses SELF-SIGNED certs instead — same Kubernetes/Traefik TLS mechanics.

Gateway API routing

CRDs are installed on this cluster already, but no GatewayClass/controller is wired up (verified next slide).
Covered as concept + comparison — a stretch lab to enable it, not assumed knowledge.

Everything else — Ingress, IngressClass, Traefik CRDs, TLS Secrets, middleware, load-balancing, egress policy — is fully real on one node.

What's actually running on this cluster

```
$ kubectl get pod -n kube-system -l app.kubernetes.io/name=traefik -o jsonpath='{...image}'  
rancher/mirrored-library-traefik:3.7.8
```

```
$ kubectl get ingressclass  
NAME                CONTROLLER                AGE  
traefik (default)   traefik.io/ingress-controller  4d
```

```
$ kubectl get svc traefik -n kube-system  
TYPE                EXTERNAL-IP                PORT(S)  
LoadBalancer        192.168.230.103            80:31803/TCP,443:31808/TCP
```

CRDs present

traefik.io/*: IngressRoute, Middleware, TLSOption, ServersTransport, ...

gateway.networking.k8s.io/*: present, but kubectl get gatewayclass -> No resources found

k3s ships Traefik ON by default — not an add-on you install; already the cluster's default IngressClass.

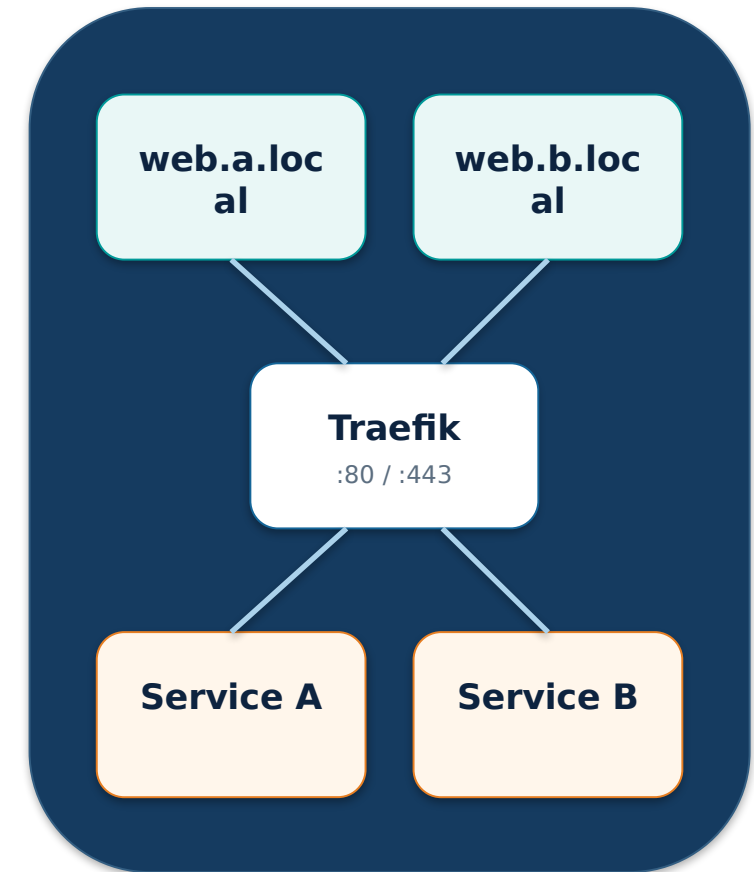
What Ingress solves (and what it doesn't)

NodePort/LoadBalancer give one Service, one port, on the node IP — ten apps need ten NodePorts.

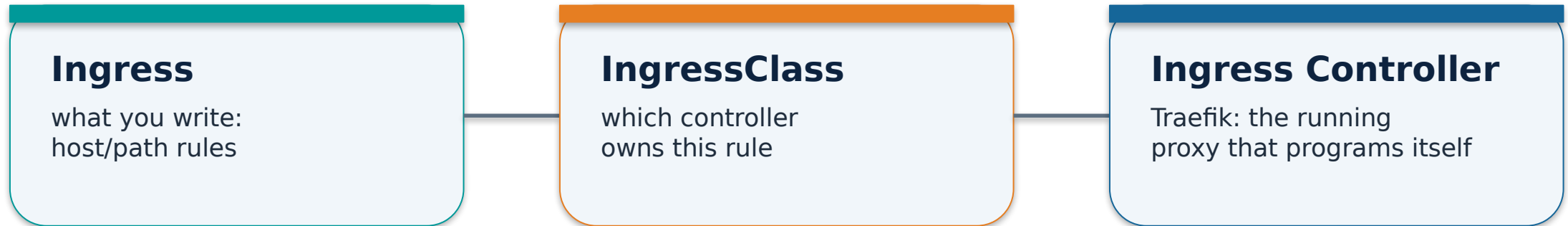
Ingress gives ONE entry point (one IP, ports 80/443) routing by hostname and/or path to many Services.

Ingress is HTTP/HTTPS (L7) routing — it reads the Host header and URL path, not raw TCP/UDP.

Ingress does not replace the Service — a Service still selects Pods; Ingress sits in front of it.



The three-part model



Key point

Writing an Ingress YAML with no controller running does NOTHING — it is a request, not a router. "traefik (default)" IngressClass means this cluster already has a controller to serve it.

Anatomy of a plain Ingress

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web
  namespace: day04-ingress
spec:
  ingressClassName: traefik
  rules:
    - host: web.k3s.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: web
                port: { number: 80 }
```

ingressClassName: explicit is better than relying on the cluster default.

host: no real public DNS here — hands-on adds this to /etc/hosts, or tests with curl --resolve.

pathType Prefix vs Exact vs ImplementationSpecific — get this wrong and two rules silently shadow each other.

Beyond plain Ingress: Traefik's own CRDs

CRD	Purpose
IngressRoute	Traefik-native alternative to Ingress; richer match rules (Host() && PathPrefix())
Middleware	redirect, strip prefix, headers, rate limit, basic auth
TLSOption	TLS version / cipher policy per endpoint
ServersTransport	how Traefik talks to the backend (e.g. skip TLS verify)
IngressRouteTCP/UDP	raw TCP/UDP routing, not HTTP

Reach for IngressRoute/Middleware when plain Ingress syntax can't express what you need — not as the default starting point.

TLS at the edge

client --TLS handshake--> Traefik (websecure :443) --plain HTTP internally--> Pod

TLS terminates AT Traefik by default — Traefik-to-Pod traffic is normal HTTP inside the cluster.

Re-encrypt with ServersTransport/mTLS only if your threat model needs it.

websecure entrypoint (:443 external, :8443 in-container here) already has http.tls=true (Slide 4).

```
spec:
  tls:
    - hosts: ["web.k3s.local"]
      secretName: web-tls

# Secret type must be:
# kubernetes.io/tls (tls.crt + tls.key)
```

Certificates: three options, one realistic here

Option	How it works	Works on this node?
Self-signed (openssl/mkcert)	you generate it; clients must trust it	YES - hands-on path
cert-manager + Let's Encrypt	auto issue/renew; needs public DNS + inbound 80/443	No - neither exists here
cert-manager + private CA	automation, works fully inside a private network	In principle - not installed, out of scope today

Don't read "self-signed" as a shortcut

The Kubernetes-side mechanics (a `tls.crt/tls.key` Secret, referenced by `spec.tls`) are IDENTICAL regardless of where the certificate came from. Only how the Secret gets populated changes.

Gateway API: what it is, and its status here

Separate GatewayClass (infra), Gateway (listeners: ports/hosts/TLS), and HTTPRoute (routing) objects.

Designed for multi-team clusters: platform team owns Gateway, app teams own their own HTTPRoute, with RBAC between them.

On THIS cluster: CRDs present, but no GatewayClass exists — Traefik is not configured as a Gateway API controller here.

That's a documented gap, not a bug — a good stretch lab, not assumed knowledge.

Rule of thumb

Ingress: straightforward single-team host/path routing you have today.

Gateway API: multiple teams sharing one entry point with real RBAC boundaries.

```
$ kubectl get gatewayclass  
No resources found
```

Load balancing at two different layers

Layer	What balances	Mechanism
Service (L4)	traffic across Pods behind one Service	kube-proxy iptables, weighted random per KUBE-SEP-* chain
Ingress/Traefik (L7)	traffic across Pods for ONE Ingress rule	Traefik's own LB: round robin (default), weighted, optional sticky sessions (cookie)

Sticky sessions matter when an app keeps in-memory session state — the cookie pins a client to the same Pod.

On one node every backend Pod is on the same host — the algorithm is 100% real and observable; only the network hop is local.

Traefik is a reverse proxy

Reverse proxy

Sits in front of SERVERS, accepts client requests, forwards to a backend on the client's behalf.

Forward proxy (contrast)

Sits in front of CLIENTS, forwards on their behalf — opposite direction. (not what Traefik does here)

Per request, Traefik:

reads Host/path and matches an Ingress/IngressRoute rule

opens or reuses a connection to a backend Pod

adds proxy headers: X-Forwarded-For, X-Forwarded-Proto, X-Forwarded-Host

streams the response back

Apps checking `request.is_secure` or building absolute URLs must trust X-Forwarded-Proto/-Host, or they "think" they're plain HTTP behind a TLS Ingress.

Host-based vs path-based routing

Host-based

web.a.local -> Service A
web.b.local -> Service B

Cleanest, no path rewriting, each app owns its own domain concept.
Needs one hostname per app.

Path-based

k3s.local/ -> Service web
k3s.local/api -> Service api

One hostname, multiple paths.
Usually needs a stripPrefix middleware so the backend doesn't see /api in its own routes.

Mixing both under one Ingress is legal and common — keep the rule set small enough that a human can still predict which rule wins.

Middleware: shaping the request

```
apiVersion: traefik.io/v1alpha1
kind: Middleware
metadata: { name: web-redirect-https }
spec:
  redirectScheme: { scheme: https, permanent: true }
---
apiVersion: traefik.io/v1alpha1
kind: Middleware
metadata: { name: api-strip-prefix }
spec:
  stripPrefix: { prefixes: ["/api"] }
```

redirectScheme — force HTTP -> HTTPS (pairs with the TLS setup).

stripPrefix — remove /api before it reaches the backend.

Others: rateLimit (abuse control),
basicAuth/forwardAuth (cheap protection),
headers (security headers, CORS).

Middlewares CHAIN — order in the Ingress annotation or IngressRoute list matters.

Egress: the other direction

Egress = traffic LEAVING a Pod toward something else — a genuinely different control point, not "Ingress in reverse."

k3s ships NO dedicated egress gateway/proxy out of the box — that's Istio/Linkerd/Cilium territory, not Flannel + Traefik + kube-router.

```
spec:
  podSelector: { matchLabels: { app: web } }
  policyTypes: [Egress]
  egress:
    - to: [{ podSelector: { matchLabels: { app: allowed-backend } } }]
    - to: [] # DNS almost always needs an explicit allow
  ports: [{ protocol: UDP, port: 53 }]
```

Classic mistake

Restricting egress and forgetting the DNS (port 53) allow rule — the Pod can't even resolve names it's allowed to reach.

Ingress best practices checklist

Always set ingressClassName explicitly — never rely on "there's only one controller."

TLS on every externally reachable host; redirect HTTP -> HTTPS rather than serving both.

Keep the Traefik dashboard (entrypoint :8080) off the public path — unauthenticated read of your routing table by default.

Prefer a few predictable host/path rules over clever overlapping ones — rule precedence is a real incident source.

Put rate limiting/auth in Middleware, not app code — visible and auditable in the routing layer.

Set resource requests/limits on the Ingress controller itself — single point of failure for every app behind it.

Rotate/monitor certificate expiry regardless of source — a self-signed cert expires exactly like a purchased one.

Hands-on labs — built and verified

Lab	Proves	Needs
ex01 basic Ingress	host-based routing to two Services, one Traefik endpoint	Ingress, --resolve or /etc/hosts
ex02 TLS	self-signed cert terminated by Traefik, HTTP->HTTPS redirect	openssl, a Middleware
ex03 path routing	one host, two paths, stripPrefix, observed on the real backend request	2 Services, 2 Ingress objects
ex04 Traefik CRDs	same routing as ONE IngressRoute, plus a 2-Middleware chain	Traefik CRDs (already installed)
ex05 egress	a Pod reaches an allowed backend + DNS, denied everything else	NetworkPolicy egress rules

Same pattern as Days 01-04: manifests + LAB-MANUAL.md (steps + troubleshooting) + OUTPUT.md (real captured output). Two real quirks found and documented: a transient HTTP 503 right after apply, and a curl -s vs -sS gotcha.

Troubleshooting an Ingress that "doesn't work"

```
kubectl get ingress -n <ns> # rule present? right class?
kubectl get ingressclass # controller actually exists?
kubectl describe ingress <name> -n <ns> # events at the bottom
kubectl -n kube-system logs deploy/traefik --tail=50
kubectl get secret <tls-secret> -n <ns> -o jsonpath='{.type}'
curl -vk https://<host> --resolve <host>:443:<node-ip> # bypass DNS entirely
```

Symptom	Likely cause
404 from Traefik itself	no rule matched - check host/path, ingressClassName
works with --resolve, not from a browser	DNS/hosts file isn't pointing the hostname at the node
TLS handshake fails / wrong cert	Secret isn't kubernetes.io/tls, or tls.hosts != request host (SNI)
two rules seem to conflict	overlapping path/host across Ingress objects
middleware seems ignored	wrong reference name/namespace, or wrong order in IngressRoute

RECAP

One entry point. Many rules. Terminate TLS once.

Ingress object = rules. IngressClass = which controller. Traefik = the controller already running here.

TLS terminates at the edge; a `kubernetes.io/tls` Secret is the same mechanic self-signed or CA-issued.

Gateway API exists here as CRDs only - a real next step, not a live lab, this round.

Load balancing happens at two layers - Service iptables, Traefik L7 - know which one you're tuning.

Egress is a different control point than Ingress, with its own NetworkPolicy rules; k3s ships no egress gateway.

Every best practice is enforceable and checkable - most of them in the troubleshooting table.